

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТЕХНИЧЕСКИХ
СИСТЕМ И ЭЛЕКТРОНИКИ

КУРСОВАЯ РАБОТА

по дисциплине
«ИНТЕРНЕТ ТЕХНОЛОГИИ»

на тему:
«Разработка веб-приложения для создания резюме.»

Выполнил:

Столяр Артем Максимович
Студент 4 курса группы ПИН-б-з-22-1
Направления подготовки
09.03.03 Прикладная информатика
заочной формы обучения

(подпись)

Руководитель работы:

Щеголев А.А.
(ФИО)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

_____	_____	_____
_____	_____	_____
_____	_____	_____

Ставрополь, 2026 г

УТВЕРЖДАЮ

директор департамента цифровых,
робототехнических систем
и электроники
_____ Азаров И.В.

Институт Перспективной инженерии
Департамент Цифровых, робототехнических систем и
электроники
Направление подготовки 09.03.03
Направленность (профиль) Прикладная информатика

ЗАДАНИЕ

на курсовую работу/проект

студента Столяр Артем Максимович
(фамилия, имя, отчество)

по дисциплине Интернет-технологии

1. Тема работы «Разработка веб-приложения для создания резюме»

2. Цель - разработать веб-приложение для создания резюме, которое позволит пользователям подстраиваться под рынок труда и создавать резюме под конкретную вакансию.

3. Задачи Сбор и анализ информации: Поиск и изучение литературы, статей, научных исследований и других источников информации по выбранной теме. Планирование работы: Составление плана курсовой работы, определение основных этапов и сроков выполнения. Написание текста: Оформление результатов исследования в виде структурированного текста, включающего введение, основную часть и заключение. Проведение экспериментов и практическая реализация: Если это необходимо, выполнение практических заданий, связанных с разработкой программного обеспечения, созданием веб-сайтов или проведением экспериментов. Оформление работы: Соблюдение требований к оформлению курсовой работы, включая правильное оформление списка литературы, таблиц, рисунков и схем. Представление и защита работы: Подготовка презентации и выступление перед комиссией, где студент демонстрирует результаты своего труда и отвечает на вопросы.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части Исторический обзор развития интернет-технологий (в зависимости от выбранной темы: стека технологий). Основные понятия и определения. Описание выбранного стека технологий. Анализ существующих решений и подходов.

б) по практической части Постановка задачи. Выбор методов и инструментов для решения задачи. Описание процесса разработки. Результаты эксперимента или практической реализации. Оценка эффективности предложенных решений.

5. Исходные данные:

а) по литературным источникам _____

б) по вариантам, разработанным преподавателем _____

в) иное _____

6. Список рекомендуемой литературы _____

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “ ____ ” _____ 20_ г.

50 % - _____ “ ____ ” _____ 20_ г.

75 % - _____ “ ____ ” _____ 20_ г.

100 % - _____ “ ____ ” _____ 20_ г.

8. Срок защиты студентом курсовой работы “ ____ ” _____ 20_ г.

Дата выдачи задания “ ____ ” _____ 20_ г.

Руководитель курсовой работы

Старший преподаватель департамента цифровых, робототехнических систем
и электроники института перспективной инженерии Щеголев А.А.

(ученая степень, звание)(личная подпись)(инициалы, фамилия)

Задание принял к исполнению студент заочной формы обучения 4 курса
ПИН-б-3-22-1 группы _____

(личная подпись)

(инициалы, фамилия)

Оглавление

ВВЕДЕНИЕ.....	5
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ.....	8
1.1 Исторический обзор развития интернет-технологий.....	8
1.2 Основные понятия веб-разработки.....	10
1.3 Описание выбранного стека технологий.....	13
1.4 Анализ существующих решений для создания резюме.....	16
2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ СОЗДАНИЯ РЕЗЮМЕ.....	20
2.1 Постановка задачи.....	20
2.2 Выбор средства создания веб-приложения.....	21
2.3 Описание объектной модели.....	24
2.4 База данных и серверная часть.....	26
2.5 Разработка клиентской части.....	28
2.6 Использование системы контроля версий, развертывание.....	31
2.7 Тестирование и верификация работоспособности.....	32
ЗАКЛЮЧЕНИЕ.....	33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	35

ВВЕДЕНИЕ

Актуальность темы обусловлена активным развитием цифровых технологий и ростом популярности онлайн-сервисов в сфере управления человеческими ресурсами и карьерного развития. В современных условиях пользователи стремятся получать доступ к необходимым сервисам быстро и удобно, не прибегая к сложным офисным программам и услугам профессиональных составителей. Одним из наиболее востребованных направлений является создание резюме через интернет, позволяющее соискателям оперативно формировать структурированные и визуально привлекательные документы, адаптированные под требования работодателей, а также получать рекомендации по их улучшению в режиме реального времени. В связи с этим разработка веб-приложений для автоматизации процесса создания резюме представляет значительный практический интерес и отвечает современным требованиям рынка труда и цифровой экономики.

Объект исследования – процесс разработки веб-приложений для автоматизации создания резюме с использованием современных веб-технологий.

Предмет исследования – методы, средства и технологии проектирования и реализации веб-приложения для генерации резюме, а также организация взаимодействия клиентской и серверной частей системы, включая обработку пользовательских данных и экспорт готовых документов.

Цель работы – разработка и документирование веб-приложения для создания резюме, обеспечивающего удобный ввод личной и профессиональной информации, выбор шаблонов оформления, предварительный просмотр и экспорт готового резюме в распространённые форматы, а также демонстрирующего практическое применение современных технологий веб-разработки.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ существующих онлайн-сервисов по созданию резюме и выявить их функциональные возможности и ограничения
- исследовать существующие решения в области каршеринга и аренды транспортных средств, определить их основные преимущества и недостатки;
- сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;
- обосновать выбор программных средств, языков программирования, библиотек и фреймворков для реализации проекта;
- разработать структуру базы данных для хранения информации об пользователях и их резюме;
- реализовать серверную часть приложения, обеспечивающую обработку запросов пользователей и управление данными;
- разработать клиентскую часть приложения с удобным пользовательским интерфейсом для создания и редактирования резюме;
- реализовать механизм просмотра своих резюме с возможностью редактирования;
- реализовать экспорт резюме в pdf формате;
- организовать взаимодействие клиентской и серверной частей посредством API;
- использовать систему контроля версий Git для отслеживания изменений в проекте и размещения исходного кода в удаленном репозитории;
- провести тестирование разработанного приложения и оценить корректность его работы.

Методы исследования: анализ научной, учебной и технической литературы, изучение документации используемых технологий, сравнительный анализ существующих веб-сервисов, методы объектно-ориентированного проектирования, программирование и тестирование программного обеспечения.

Теоретическая значимость работы заключается в систематизации и обобщении знаний о современных подходах к разработке веб-приложений, архитектуре клиент-серверных систем, принципах построения информационных сервисов и организации процессов создания резюме.

Практическая значимость работы заключается в создании полнофункционального веб-приложения для генерации резюме, которое может быть использовано как самостоятельный готовый продукт для конечных пользователей, так и в качестве масштабируемой основы для последующей разработки коммерческого сервиса в сфере HR-tech. Кроме того, разработанное приложение представляет собой наглядный учебный пример, демонстрирующий интеграцию современных клиентских и серверных технологий, принципы проектирования пользовательских интерфейсов и работу с данными. Полученные в ходе выполнения работы результаты обладают потенциалом для дальнейшего расширения за счет внедрения дополнительных функциональных возможностей, таких как: внедрение ИИ-ассистента, размещение базы вакансий с возможностью подбора резюме под конкретные требования работодателя, интеграция с API популярных платформ по поиску работы, разделение пользователей на роли («Соискатель», «Рекрутер»)

Структура работы соответствует логике исследования и включает введение, две главы, заключение, список использованных источников и приложения. В первой главе рассматриваются теоретические аспекты разработки веб-приложений, анализируются современные технологии и существующие сервисы, а также обосновывается выбор программных средств для реализации проекта. Во второй главе подробно описываются этапы практической разработки системы: постановка задачи, проектирование структуры приложения и базы данных, реализация серверной и клиентской частей, настройка взаимодействия компонентов системы, тестирование и анализ результатов. В заключении подводятся итоги выполненной работы,

формулируются основные выводы и определяются перспективы дальнейшего развития разработанного веб-приложения.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ

1.1 Исторический обзор развития интернет-технологий

Развитие интернет-технологий выступает одним из ключевых драйверов цифровой трансформации всех сфер общественной жизни. Возникновение глобальной сети Интернет и создание Всемирной паутины (World Wide Web) в начале 1990-х годов ознаменовали новый этап в эволюции информационного обмена, устранив географические барьеры между пользователями. Фундаментальные основы современной веб-разработки были заложены Тимом Бернерсом-Ли, предложившим концепцию гипертекстовой системы, включающую язык разметки HTML, протокол передачи данных HTTP и универсальную систему адресации ресурсов URL.

На начальном этапе веб-ресурсы представляли собой статические страницы, содержимое которых формировалось заранее и не изменялось в ответ на действия пользователя. Основное назначение таких сайтов сводилось к публикации текстовой информации и обеспечению навигации между связанными документами. Несмотря на функциональную ограниченность, данный подход заложил фундамент для дальнейшей эволюции сети.

Переломным моментом стало появление технологий динамического формирования веб-страниц. Ключевую роль в этом процессе сыграл язык программирования JavaScript, представленный в 1995 году. Его внедрение позволило исполнять программный код непосредственно в браузере, обрабатывать пользовательские события и модифицировать содержимое страниц без их перезагрузки. Последующее развитие технологий асинхронного обмена данными (AJAX) обеспечило качественно новый уровень интерактивности веб-приложений, приблизив их по удобству к настольным программам.

Одновременно с клиентскими технологиями совершенствовалась серверная инфраструктура. Для обработки пользовательских запросов стали применяться специализированные языки программирования и платформы, обеспечивающие генерацию динамического контента и взаимодействие с системами управления базами данных. Это открыло возможность создания многофункциональных информационных систем с поддержкой регистрации, хранения персональных данных, обработки заказов и реализации сложных бизнес-логик.

Важным этапом эволюции стало распространение клиент-серверной архитектуры, предусматривающей четкое разделение функций отображения информации и обработки данных между клиентской и серверной частями. Такое структурное разделение повысило масштабируемость, облегчило сопровождение программных продуктов и упростило внесение изменений. Впоследствии широкое распространение получили программные интерфейсы приложений (API), обеспечивающие стандартизированное взаимодействие между разнородными компонентами информационных систем.

С наступлением XXI века рост вычислительных мощностей и пропускной способности сетей обусловил переход к высокопроизводительным веб-приложениям с расширенным функционалом. Веб-сайты эволюционировали в полноценные программные комплексы, охватывающие электронную коммерцию, сервисы бронирования, корпоративное управление, дистанционное образование и множество других областей.

Особого внимания заслуживает трансформация архитектурных подходов к разработке программного обеспечения. Если ранее доминировала монолитная архитектура, предполагающая создание приложения как единого неделимого модуля, то в настоящее время всё большую популярность приобретают микросервисные решения. Данная парадигма предполагает декомпозицию приложения на набор независимых сервисов, каждый из которых отвечает за отдельную бизнес-функцию. Такой подход упрощает

сопровождение, повышает отказоустойчивость системы и обеспечивает гибкое масштабирование отдельных компонентов в зависимости от нагрузки.

Современный ландшафт веб-разработки характеризуется активным применением специализированных фреймворков, распределенных систем управления базами данных, технологий контейнеризации (Docker) и средств непрерывной интеграции и развертывания (CI/CD). Для построения пользовательских интерфейсов широко используются современные JavaScript-библиотеки и фреймворки (React, Vue, Angular), обеспечивающие высокую производительность и эргономичность взаимодействия. Серверная логика реализуется с применением платформ и архитектурных решений, ориентированных на обеспечение надежности, информационной безопасности и горизонтального масштабирования.

Таким образом, эволюция интернет-технологий прошла путь от простейших статических гипертекстовых страниц до сложных распределенных информационных систем, способных обслуживать многомиллионную пользовательскую аудиторию. Современный инструментарий веб-разработки открывает широкие перспективы для создания высокопроизводительных сервисов, в том числе систем для создания резюме, позволяющих автоматизировать кадровые процессы и предоставлять пользователям качественный сервис на уровне ведущих мировых стандартов

1.2 Основные понятия веб-разработки

Для понимания принципов построения современных информационных систем необходимо рассмотреть базовые понятия и терминологию, используемую в области веб-разработки.

Веб-приложение представляет собой программный комплекс, функционирующий в сетевой среде и обеспечивающий взаимодействие пользователя с информационными ресурсами посредством веб-браузера. Ключевое отличие веб-приложений от традиционных настольных программ заключается в отсутствии необходимости установки на устройство конечного

пользователя — доступ к функционалу обеспечивается с любой платформы, поддерживающей современный браузер и имеющей подключение к сети Интернет.

Архитектурной основой подавляющего большинства веб-приложений выступает клиент-серверная модель, предполагающая разделение системы на две логические части: клиентскую и серверную. Клиентская часть отвечает за визуализацию пользовательского интерфейса и обработку интерактивных действий, тогда как серверная часть реализует бизнес-логику, управляет хранимыми данными и обрабатывает входящие запросы, формируя ответы для клиента.

Клиентская часть (Frontend) представляет собой код, исполняемый в среде веб-браузера. Её функциональное назначение включает отображение информации, обеспечение эргономичного и отзывчивого интерфейса, а также реакцию на действия пользователя. В основе разработки клиентской части лежат три фундаментальные технологии:

- HTML (HyperText Markup Language) — язык гипертекстовой разметки, определяющий структуру и семантическое содержание веб-документов;

- CSS (Cascading Style Sheets) — каскадные таблицы стилей, предназначенные для визуального оформления элементов интерфейса и обеспечения адаптивной вёрстки под различные размеры экранов;

- JavaScript — язык программирования высокого уровня, обеспечивающий интерактивность веб-страниц, динамическое обновление содержимого и асинхронное взаимодействие с сервером без перезагрузки страницы.

Серверная часть (Backend) — это программное обеспечение, функционирующее на серверной стороне и выполняющее обработку запросов, поступающих от клиентов. К числу основных функций серверной части относятся: реализация прикладной бизнес-логики, валидация и обработка пользовательских данных, взаимодействие с системами хранения

информации, обеспечение безопасности, а также формирование и передача клиенту структурированных ответов (как правило, в форматах JSON или XML).

Коммуникация между клиентской и серверной частями осуществляется посредством протокола HTTP — прикладного протокола передачи гипертекстовых документов. Каждый HTTP-запрос содержит метод обращения, URI-адрес ресурса и набор заголовков с метаданными. В ответ сервер направляет клиенту запрошенные данные или служебное сообщение вместе с кодом состояния, отражающим успешность или характер ошибки выполнения операции.

Наиболее распространённые методы HTTP-запросов включают:

- GET — получение существующих данных без изменения состояния ресурса;
- POST — создание нового ресурса (например, добавление записи в базу);
- PUT — полная замена существующего ресурса;
- PATCH — частичное обновление данных ресурса;
- DELETE — удаление ресурса.

Для стандартизированного взаимодействия между различными программными компонентами и сторонними системами применяются программные интерфейсы приложений (API). API определяет формализованный набор правил, форматов данных и методов вызова, позволяющий разнородным системам обмениваться информацией и интегрироваться друг с другом. Наибольшее распространение в веб-разработке получил архитектурный стиль *REST*, который опирается на стандартные возможности протокола HTTP и трактует каждый ресурс как уникальный URL-адрес, доступный для выполнения операций посредством вышеперечисленных методов.

Критически важным компонентом большинства веб-приложений являются системы управления базами данных (СУБД), предназначенные для

надёжного долговременного хранения, структурирования и оперативного извлечения информации. В зависимости от характера данных и требований к производительности могут применяться реляционные СУБД (основанные на табличных моделях и языке SQL) или нереляционные (NoSQL-решения), обеспечивающие гибкую схему хранения и горизонтальное масштабирование.

Обеспечение информационной безопасности веб-приложений базируется на механизмах аутентификации и авторизации. Аутентификация представляет собой процесс проверки подлинности пользователя (как правило, по паре «логин/пароль» или с использованием токенов), тогда как авторизация определяет множество разрешённых операций и доступных ресурсов для уже идентифицированного пользователя. Реализация данных механизмов является обязательным условием защиты конфиденциальных персональных данных и предотвращения несанкционированного доступа.

Таким образом, рассмотренные фундаментальные понятия — клиент-серверная архитектура, HTTP-протокол, API, СУБД и системы безопасности — образуют концептуальный базис для разработки веб-приложений любого назначения. В контексте данной работы перечисленные технологии и подходы будут применены для создания функционального веб-сервиса по генерации резюме, где клиентская часть обеспечит удобный интерфейс для ввода и редактирования данных, серверная часть реализует логику формирования готового документа и управления пользовательскими профилями, а база данных выступит средством хранения персональной информации и шаблонов оформления.

1.3 Описание выбранного стека технологий

Для реализации веб-приложения по созданию резюме был подобран современный технологический стек, отвечающий требованиям производительности, масштабируемости и удобства дальнейшего сопровождения программного продукта. Критериями выбора инструментов выступили надёжность системы, потенциал для расширения

функциональности, а также соответствие актуальным промышленным стандартам и методологиям разработки.

В качестве серверной платформы применён NestJS — прогрессивный фреймворк для построения серверных приложений на языке TypeScript. Ключевым достоинством NestJS является модульная архитектура, позволяющая структурировать систему в виде независимых функциональных блоков, что обеспечивает высокую связанность внутри модулей и слабое зацепление между ними. Фреймворк предоставляет встроенные механизмы внедрения зависимостей, поддержку объектно-ориентированного и функционального программирования, а также реализацию распространённых архитектурных паттернов (контроллеры, сервисы, репозитории), что сближает процесс разработки с корпоративными стандартами.

Выбор TypeScript в качестве основного языка программирования обусловлен его статической типизацией, которая существенно повышает надёжность программного обеспечения за счёт выявления ошибок на этапе компиляции, упрощает навигацию по кодовой базе и облегчает рефакторинг. По сравнению с динамической типизацией JavaScript, TypeScript обеспечивает большую предсказуемость поведения системы и снижает порог входа для новых участников команды.

Для взаимодействия с базой данных используется технология ORM (Object-Relational Mapping), позволяющая оперировать сущностями предметной области через объектно-ориентированный интерфейс без необходимости ручного написания SQL-запросов. ORM-слой абстрагирует разработчика от особенностей конкретной СУБД, ускоряет написание типовых операций CRUD и способствует поддержанию единообразия при работе с данными на всех уровнях приложения.

Безопасность приложения обеспечивается механизмами аутентификации и авторизации на основе JSON Web Token (JWT). Токен-ориентированный подход позволяет идентифицировать пользователя без сохранения состояния на сервере, что особенно важно для горизонтального

масштабирования. JWT-токены содержат цифровую подпись, гарантирующую целостность и подлинность передаваемых данных, а также обеспечивают гибкое разграничение прав доступа к различным функциональным модулям системы.

Клиентская часть разработана с применением Vue.js — прогрессивного JavaScript-фреймворка для построения пользовательских интерфейсов. Выбор Vue.js обоснован его компонентно-ориентированной архитектурой, высокой производительностью виртуального DOM, а также низким порогом входа и обширной экосистемой инструментов. Фреймворк позволяет эффективно реализовывать реактивные интерфейсы с минимальными накладными расходами на обновление представления при изменении данных.

При проектировании клиентской части принята архитектурная методология Feature-Sliced Design (FSD). Данный подход обеспечивает чёткое разделение ответственности между слоями, упрощает навигацию по проекту и способствует переиспользованию компонентов. FSD особенно эффективна при росте размера приложения, поскольку предотвращает хаотичное разрастание кода и облегчает командную разработку.

Коммуникация между клиентской и серверной частями организована посредством REST API — архитектурного стиля, использующего стандартные HTTP-методы для выполнения операций над ресурсами. Выбор REST обусловлен его универсальностью, простотой реализации, независимостью от клиентских платформ и широкой поддержкой в экосистеме веб-технологий. Обмен данными осуществляется в формате JSON, обеспечивающем лёгкость парсинга и читаемости.

Развёртывание программного комплекса выполняется с применением Docker — платформы контейнеризации, позволяющей упаковывать приложения и их зависимости в изолированные контейнеры. Контейнеризация решает проблему несовместимости окружений на этапах разработки, тестирования и эксплуатации, обеспечивая воспроизводимость сборки. Каждый компонент системы (серверное приложение, база данных, клиентский

сборщик) функционирует в отдельном контейнере, что упрощает управление инфраструктурой и повышает переносимость продукта между различными хостинговыми платформами.

Для автоматизации процессов сборки, тестирования и доставки внедрена методология CI/CD. Настроенные пайплайны выполняют линтинг, прогон модульных и интеграционных тестов, а также сборку Docker-образов при каждом изменении в репозитории. Автоматизация минимизирует человеческий фактор, ускоряет выпуск релизов и гарантирует стабильность основного бранча.

Контроль версий реализован с использованием распределённой системы Git. Для хранения удалённого репозитория и организации совместной работы задействована платформа GitHub, предоставляющая инструменты для управления задачами, рецензирования кода и непрерывной интеграции. Использование GitHub позволяет вести прозрачную историю изменений, документировать принятые решения и обеспечивать коллаборацию в рамках проектной деятельности.

Таким образом, представленный технологический стек полностью соответствует современным стандартам веб-разработки и обеспечивает создание надёжной, масштабируемой и удобной в сопровождении системы для генерации резюме. Выбранные инструменты и подходы ориентированы не только на решение текущих задач, но и на долгосрочное развитие проекта с минимальными затратами на переработку архитектуры.

1.4 Анализ существующих решений для создания резюме

В настоящее время рынок HR-технологий и сервисов по трудоустройству активно развивается, что обусловлено цифровизацией кадровых процессов и ростом потребности соискателей в эффективных инструментах для поиска работы. Современные платформы позволяют пользователям создавать профессиональные резюме, отслеживать вакансии, взаимодействовать с работодателями и получать рекомендации по карьерному

развитию в режиме онлайн. Рассмотрим наиболее распространённые решения, представленные на российском и международном рынках.

HeadHunter — крупнейшая российская платформа для поиска работы и подбора персонала. Сервис предоставляет пользователям функциональный конструктор резюме с возможностью заполнения всех ключевых разделов: опыт работы, образование, навыки, дополнительные сведения. Платформа предлагает автоматические подсказки при заполнении, рекомендации по улучшению резюме и оценку его видимости для работодателей. К достоинствам сервиса относятся широкая база вакансий, развитая система фильтрации и персонализированные рекомендации. Недостатками можно считать ограниченный выбор шаблонов оформления и платный доступ к некоторым расширенным функциям.

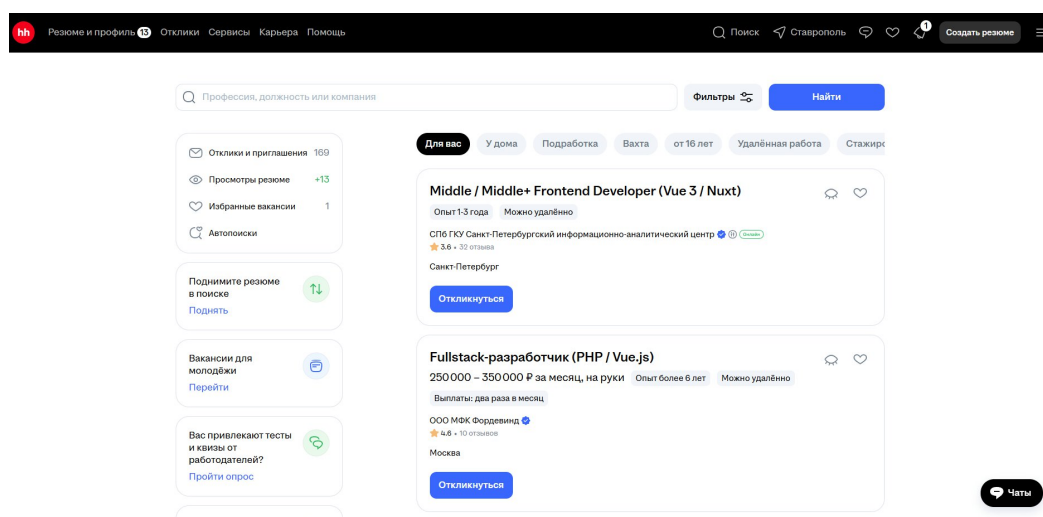


Рисунок 1.1 — Интерфейс сервиса «HeadHunter»

Работа.ру — один из старейших российских порталов по трудоустройству, предоставляющий классический набор функций для соискателей и работодателей. Сервис позволяет создавать резюме с базовым набором разделов, просматривать вакансии и отправлять отклики. К преимуществам относится многолетняя репутация и широкая база работодателей. Среди недостатков — устаревающий интерфейс и отсутствие современных инструментов для автоматического форматирования и экспорта резюме в различные форматы.

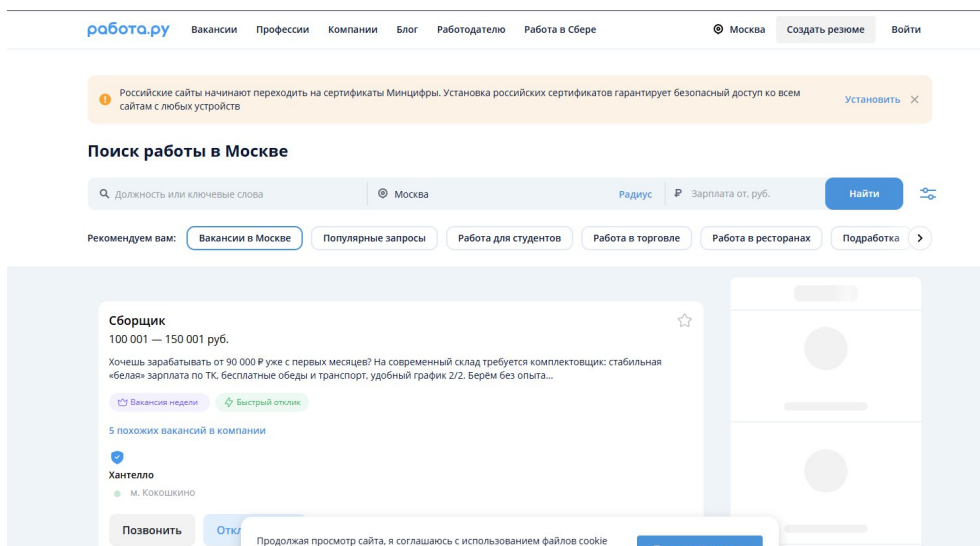


Рисунок 1.2 — Интерфейс сервиса «Работа.ру»

Habr Карьера — специализированная платформа для IT-специалистов, позволяющая создавать резюме с акцентом на технические навыки и проектный опыт. Сервис предлагает интеграцию с GitHub и другими профильными ресурсами, что удобно для разработчиков. К преимуществам относятся узкая специализация (IT-сфера) и наличие технического сообщества. Недостатком является ограниченный охват вакансий из других профессиональных областей

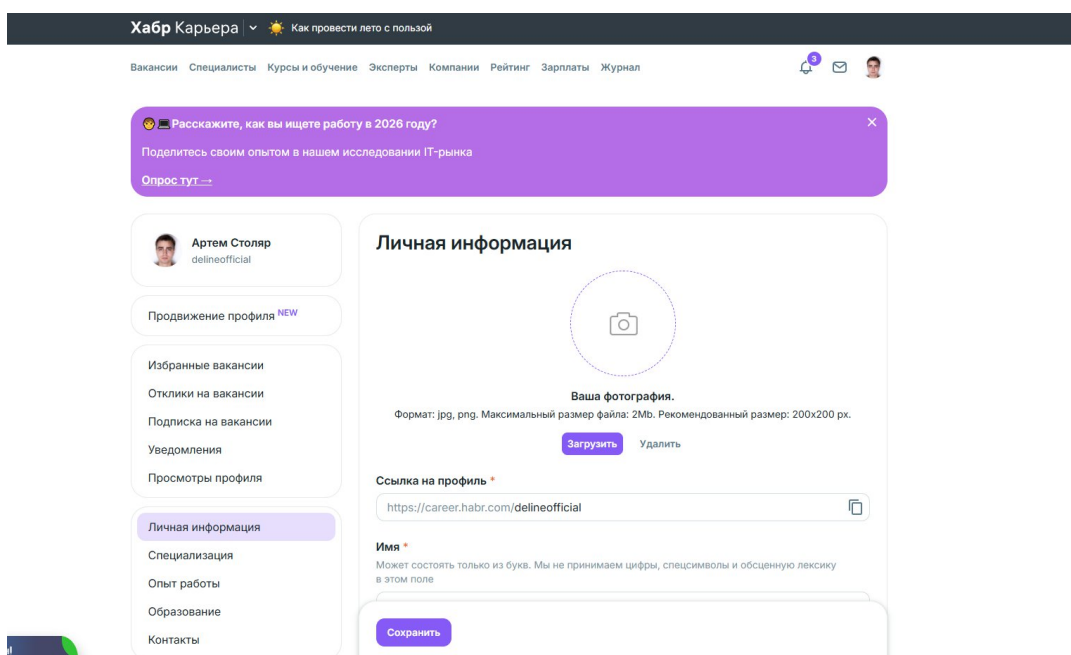


Рисунок 1.3 — Интерфейс сервиса «Habr Карьера»

Помимо общих платформ для поиска работы, существуют узкоспециализированные конструкторы резюме, ориентированные исключительно на создание и оформление документа. Такие платформы предлагают пользователям широкий выбор визуальных шаблонов, возможность перетаскивания блоков (drag-and-drop), а также экспорт в PDF и DOCX. Их основным преимуществом является качественное визуальное оформление и простота использования, однако они не предоставляют базы вакансий и возможности прямого взаимодействия с работодателями.

Проведённый анализ показал, что большинство существующих решений обладают широким функционалом, включающим конструктор резюме, каталог вакансий, систему поиска и фильтрации, отклики на вакансии, работу с пользовательскими аккаунтами и настройки приватности. Вместе с тем многие коммерческие системы перегружены дополнительными платными функциями (премиум-подписки, продвижение резюме, доступ к статистике), что усложняет интерфейс и создаёт барьеры для пользователей с ограниченным бюджетом. Кроме того, большинство платформ предлагают шаблоны резюме с ограниченными возможностями кастомизации, что не всегда позволяет соискателю выделиться среди конкурентов.

Разрабатываемое веб-приложение ориентировано на реализацию ключевых функций сервиса по созданию резюме: ввод и редактирование персональных и профессиональных данных, выбор шаблона оформления, предварительный просмотр готового документа, экспорт в распространённые форматы (PDF, DOCX), а также хранение нескольких версий резюме в личном кабинете пользователя. Такой подход позволяет сосредоточиться на изучении современных технологий веб-разработки, принципов проектирования пользовательских интерфейсов и организации взаимодействия между клиентской и серверной частями приложения.

2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ СОЗДАНИЯ РЕЗЮМЕ

2.1 Постановка задачи

На основании проведённого анализа предметной области была сформулирована задача разработки веб-приложения для создания резюме, обеспечивающего взаимодействие пользователей с каталогом своих резюме и системой создания и редактирования их через единый пользовательский интерфейс.

Разрабатываемая система должна предоставлять пользователям следующие функциональные возможности:

- возможность зарегистрироваться и залогиниться пользователю
- получение списка своих резюме с полным описанием каждого из них;
- создание своего резюме;
- возможность редактирования уже имеющегося резюме и возможностью добавить новые места работы;
- возможность загрузки резюме в формате PDF;

Приложение должно быть реализовано в соответствии с клиент-серверной архитектурой. Пользовательский интерфейс взаимодействует с серверной частью посредством программного интерфейса REST API, обеспечивающего передачу данных между компонентами системы. Серверная часть отвечает за обработку запросов, выполнение бизнес-логики, управление данными и обеспечение безопасности приложения.

При разработке системы необходимо обеспечить выполнение следующих дополнительных требований:

- валидацию входных данных как на стороне клиента, так и на стороне сервера;
- корректную обработку исключительных ситуаций и отображение информативных сообщений об ошибках;
- соблюдение принципов модульности, масштабируемости и сопровождаемости программного кода;

- использование системы контроля версий Git для отслеживания изменений проекта и организации процесса разработки.

Таким образом, результатом выполнения работы должно стать веб-приложение для создания резюме, позволяющее пользователям создавать, редактировать и экспортировать свое резюме.

2.2 Выбор средства создания веб-приложения

Выбор технологий для разработки веб-приложения осуществлялся на основе комплекса критериев, включающих производительность, масштабируемость, безопасность, удобство сопровождения и соответствие современным индустриальным стандартам. Принятые решения ориентированы на реализацию функционального сервиса по созданию резюме с применением актуальных архитектурных подходов и инструментов, обеспечивающих высокое качество конечного продукта.

Серверная часть. В качестве основы для разработки серверного компонента выбран фреймворк NestJS, построенный на платформе Node.js и предназначенный для создания масштабируемых корпоративных приложений. Ключевыми факторами выбора NestJS стали:

- модульная архитектура, допускающая декомпозицию системы на слабосвязанные функциональные блоки;
- встроенный контейнер внедрения зависимостей, упрощающий тестирование, модификацию и сопровождение кода;
- высокая структурированность проекта по сравнению с минималистичными фреймворками
- поддержка объектно-ориентированного, функционального и реактивного стилей программирования;
- обширная документация и активное сообщество разработчиков.

Разработка ведётся на языке TypeScript, являющемся типизированным надмножеством JavaScript. Статическая система типов позволяет выявлять ошибки на этапе компиляции, повышает предсказуемость поведения системы, улучшает читаемость кода и облегчает навигацию по проекту при его росте. Применение

TypeScript особенно актуально для долгосрочно сопровождаемых проектов, где важна поддержка строгой дисциплины типов.

Клиентская часть. Для реализации пользовательского интерфейса выбран прогрессивный фреймворк Vue.js. Данное решение обосновано следующими преимуществами:

- компонентный подход, обеспечивающий декомпозицию интерфейса на переиспользуемые и изолированные элементы;
- высокая производительность благодаря оптимизированному механизму виртуального DOM;
- реактивная модель данных, упрощающая синхронизацию состояния интерфейса с бизнес-логикой;
- развитая экосистема инструментов (Vue Router, Pinia, Vite) для построения полноценных SPA-приложений;
- низкий порог входа и детализированная документация, ускоряющие процесс разработки.

Для управления глобальным состоянием приложения задействовано централизованное хранилище данных, обеспечивающее предсказуемый поток информации между компонентами и упрощающее отладку. Это позволяет избежать проблем, связанных с «prop drilling» и неконтролируемым изменением состояния.

Структура клиентской части организована в соответствии с методологией Feature-Sliced Design (FSD), которая предполагает разделение кодовой базы по функциональным срезам (фичам) и уровням ответственности (сущности, виджеты, страницы, приложение). Данный подход способствует логической изоляции функциональности, упрощает навигацию по проекту и облегчает масштабирование клиентской части по мере добавления новых возможностей.

Инфраструктура и развёртывание. Для обеспечения единообразия сред разработки, тестирования и эксплуатации применяется платформа контейнеризации Docker. Контейнеризация предоставляет следующие преимущества:

- изоляция приложения от системных зависимостей и конфигурации хостовой ОС;

- воспроизводимость сборок в любом окружении;
- упрощение развёртывания за счёт упаковки всех компонентов (сервер, база данных, клиентский сервер) в стандартизированные образы;
- повышение переносимости приложения между различными хостинговыми платформами.

Для локального запуска и оркестрации многоконтейнерной инфраструктуры используется Docker Compose, позволяющий описать все сервисы проекта в едином YAML-файле и управлять ими посредством одной команды.

Управление версиями и автоматизация. Для отслеживания изменений исходного кода применяется распределённая система контроля версий Git. Её использование обеспечивает:

- сохранение полной истории изменений с возможностью отката к любой предыдущей версии;
- поддержку параллельной разработки нескольких функциональных веток;
- безопасное экспериментирование без риска нарушить стабильность основной кодовой базы;
- упрощение командной работы посредством механизмов слияния и разрешения конфликтов.

Удалённое хранение репозитория организовано на платформе GitHub, которая также служит инфраструктурной основой для автоматизации процессов сборки и развёртывания. Для реализации CI/CD-пайплайнов задействованы GitHub Actions, обеспечивающие автоматический запуск линтеров, выполнение тестов, сборку Docker-образов и деплой приложения при каждом изменении в репозитории. Автоматизация минимизирует влияние человеческого фактора, сокращает время выхода релизов и гарантирует стабильность процесса разработки.

Таким образом, представленный технологический стек в полной мере соответствует современным требованиям к веб-разработке и обеспечивает создание надёжного, масштабируемого и удобного в сопровождении веб-приложения для генерации резюме. Выбранные инструменты и подходы ориентированы на

долгосрочное развитие проекта с минимальными затратами на рефакторинг и перестройку архитектуры.

2.3 Описание объектной модели

Объектная модель приложения построена на основе микросервисной архитектуры и включает две основные сущности: User (пользователь) и Resume (резюме). Взаимодействие между клиентом и микросервисами осуществляется через единый API Gateway, который предоставляет REST API для каждой сущности.

Таблица 1.1 – Структура сущности User

Поле	Тип данных	Описание
Id	String UUID	уникальный идентификатор пользователя, генерируется автоматически;
Email	String	логин пользователя
Password	String	Хешированный пароль пользователя

Таблица 1.2 – Структура сущности Resume

Поле	Тип данных	Описание
Id	String UUID	уникальный идентификатор резюме, генерируется автоматически
fullName	String	ФИО
dateOfBirth	String	Дата рождения
title	String	Должность
summary	String	О себе
skills	String	Навыки
experiences	String	Опыт работы
photo	String	Фото
createdAt	String	Дата создания записи
userId	String UUID	Идентификатор пользователя

Для управления данными каждой из сущностей предусмотрены операции, соответствующие CRUD-паттерну:

- Create — создание новой записи (пользователя, резюме);
- Read — чтение одной записи или списка записей;
- Update — обновление существующей записи;
- Delete — удаление записи (для резюме).

Эти операции реализуются через REST API, который предоставляет следующие основные эндпоинты.

Сервис аутентификации:

- POST /api/auth/register —Регистрация нового пользователя
- POST /api/auth/login — Вход пользователя
- POST /api/auth/refresh — Обновление токенов (refresh token)
- POST /api/auth/logout — Выход пользователя

Сервис резюме:

- POST /api/resumes — Создание нового резюме (с загрузкой фото)
- GET /api/resumes/:id — Получение резюме по ID
- PUT /api/resumes/:id — Обновление резюме (с загрузкой фото)
- GET /api/users/:id/resumes — Получение всех резюме пользователя
- DELETE /api/resumes/:id —Удаление резюме

Сервер возвращает соответствующие HTTP-статусы:

- 200 OK — успешный GET, PUT или PATCH;
- 201 Created — успешное создание ресурса;
- 204 No Content — успешное удаление ресурса;
- 400 Bad Request — отсутствуют обязательные поля или данные не прошли валидацию;
- 401 Unauthorized — пользователь не авторизован или токен недействителен;
- 403 Forbidden — у пользователя недостаточно прав для выполнения операции (например, действие доступно только администратору);
- 404 Not Found — запрашиваемый ресурс с указанным id не найден;

– 500 Internal Server Error — внутренняя ошибка сервера.

Такая система статусов позволяет клиенту корректно обрабатывать ответы и информировать пользователя о результате выполнения операции.

2.4 База данных и серверная часть

Серверная часть реализована как монолитное приложение на базе NestJS с прямым подключением к локальной базе данных SQLite. В отличие от микросервисной архитектуры, все модули (auth, users, resumes) работают в едином процессе приложения, обращаясь к единой схеме базы данных через ORM TypeORM.

Запуск приложения осуществляется в файле `main.ts`: создаётся экземпляр `NestExpressApplication`, который запускает HTTP-сервер на порту 3000 (или значение из переменной окружения `PORT`). Сервер слушает подключения со всех интерфейсов (0.0.0.0), что позволяет ему работать корректно как при локальной разработке, так и внутри контейнера Docker.

В файле `main.ts` подключены следующие `middleware` и настройки:

- `app.useStaticAssets()` — сервирование загруженных фотографий из директории `/uploads` с префиксом `/api/uploads/`;
- `app.setGlobalPrefix('api')` — все маршруты доступны с префиксом `/api`;
- глобальный `ValidationPipe` — валидирует входные данные на основе DTO, отбрасывает лишние поля и преобразует тело запроса к нужному типу.

Поток обработки запроса выглядит следующим образом: клиент отправляет HTTP-запрос на REST-контроллер (`AuthController`, `ResumesController` или `AppController`), контроллер выполняет базовую валидацию через DTO и декораторы `class-validator` (`@IsString`, `@IsNotEmpty`, `@IsEmail` и др.), затем делегирует бизнес-логику соответствующему сервису (`AuthService`, `ResumesService`, `UsersService`). Сервис выполняет операции с базой данных через репозитории `TypeORM` и возвращает результат обратно контроллеру, который отправляет ответ клиенту.

Авторизация реализована через JWT-токены: при успешной авторизации `AuthService` выдаёт пару токенов (`access` и `refresh`), клиент

сохраняет их и отправляет в заголовке Authorization при каждом запросе. На защищённых маршрутах используется декоратор `@UseGuards(AuthGuard('jwt'))`, который проверяет валидность токена через стратегию `JwtStrategy` и извлекает информацию о пользователе в объект `req.user`.

Контроль доступа реализован на уровне контроллеров и сервисов: при запросе резюме система проверяет, что текущий пользователь является владельцем этого резюме, обращаясь к полю `resume.user.id` и сравнивая его с `req.user.userId`. Попытка доступа к чужому ресурсу возвращает исключение `ForbiddenException` с HTTP-статусом 403.

Загрузка файлов (фотографии профиля) реализована через `FileInterceptor` из библиотеки `@nestjs/platform-express`: при создании или обновлении резюме можно прикрепить изображение, которое сохраняется в директории `uploads` с случайным именем, а путь к файлу сохраняется в поле `data.photo` резюме. Валидация типа файла (только `jpg`, `jpeg`, `png`, `gif`, `webp`) выполняется на уровне `multer fileFilter`.

Обработка ошибок реализована через стандартные исключения NestJS (`HttpException`, `NotFoundException`, `ForbiddenException`, `UnauthorizedException`), которые автоматически преобразуются в HTTP-ответы с соответствующим статусом и сообщением об ошибке. При валидации входных данных контроллер возвращает 400 Bad Request с описанием ошибок валидации.

Листинг 1.1 – Пример реализации ошибок в `resumes.controller.ts`

```
@UseGuards(AuthGuard('jwt'))
@Get('users/:id/resumes')
async getByUser(@Request() req: any, @Param('id') id: string) {
  const userId = Number(id);
  if (req.user.userId !== userId) {
    throw new ForbiddenException('Access denied');
  }
  return this.resumesService.findById(userId);
}

@UseGuards(AuthGuard('jwt'))
@Delete('resumes/:id')
async delete(@Request() req: any, @Param('id') id: string) {
  const resumeId = Number(id);
  const resume = await this.resumesService.findById(resumeId);
```

```

    if (!resume) {
      throw new ForbiddenException('Resume not found');
    }

    if (resume.user.id !== req.user.userId) {
      throw new ForbiddenException('Access denied');
    }

    return this.resumesService.delete(resumeId);
  }
}

```

После разработки все эндпоинты серверной части были протестированы локально в контейнере Docker (`docker-compose up`): выполнены запросы регистрации и авторизации пользователя, создания резюме с загрузкой фотографии, получения и обновления резюме, удаления резюме.

2.5 Разработка клиентской части

Клиентская часть реализована на Vue 3 с TypeScript и собирается с помощью Vite. Код организован в папке `src` и разделён на несколько ключевых директорий:

- `app` — инициализация приложения: точка входа `main.ts`, корневой компонент `App.vue`, настройка роутера и глобальных стилей;
- `pages` — страницы-маршруты, компоненты для отдельных экранов приложения
- `components` — переиспользуемые UI-компоненты и элементы интерфейса;
- `composables` — логика и состояние, вынесенные в переиспользуемые хуки (`useAuth`);
- `services` — сервисы взаимодействия с сервером (API-клиент).

Приложение построено с использованием Vue 3 Composition API, что позволяет организовать логику компонентов максимально гибко и переиспользовать куски кода между различными компонентами через `composables`. Каждая страница реализована как однофайловый компонент Vue с использованием синтаксиса `<script setup>`, объединяющий разметку (`template`), логику (`script`) и стили (`style`) в одном файле.

Маршрутизация настроена в router/index.ts с помощью vue-router. Определены следующие основные маршруты:

- / — главная страница (перенаправляет на Dashboard для авторизованных пользователей);
- /login — страница входа, доступна только для неавторизованных пользователей;
- /register — страница регистрации, доступна только для неавторизованных пользователей;
- /dashboard — личный кабинет, список всех резюме пользователя (требуется авторизации);
- /create — создание нового резюме (требуется авторизации);
- /resumes/:id/edit — редактирование существующего резюме с передачей ID в параметре маршрута (требуется авторизации).

Листинг 1.1 – Реализация маршрутизации на клиенте

```
const auth = useAuth()

const routes = [
  { path: '/', name: 'Home', component: Home },
  { path: '/login', name: 'Login', component: () => import('../pages/Login.vue'), meta: { guest: true } },
  { path: '/register', name: 'Register', component: () => import('../pages/Register.vue'), meta: { guest: true } },
  { path: '/create', name: 'Create', component: () => import('../pages/CreateResume.vue'), meta: { requiresAuth: true } },
  { path: '/resumes/:id/edit', name: 'EditResume', component: () => import('../pages/EditResume.vue'), meta: { requiresAuth: true } },
  { path: '/dashboard', name: 'Dashboard', component: () => import('../pages/Dashboard.vue'), meta: { requiresAuth: true } },
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

router.beforeEach((to, from, next) => {
  if (to.meta.requiresAuth && !auth.isAuthenticated.value) {
    return next({ name: 'Home' })
  }
})
```

```

if (to.meta.guest && auth.isAuthenticated.value) {
  return next({ name: 'Dashboard' })
}

next()
})

```

На уровне навигационного хука `beforeEach` реализованы охранники маршрутов: `requiresAuth` закрывает доступ неавторизованным пользователям к приватным страницам (при попытке доступа происходит редирект на главную страницу), а `guest` скрывает страницы входа и регистрации от уже авторизованных пользователей, перенаправляя их на `Dashboard`.

Ключевые страницы:

– `LoginPage` — форма входа пользователя. Содержит два поля (электронная почта и пароль) и кнопку отправки. При отправке формы проверяется целостность данных (оба поля обязательны, email должен быть в корректном формате). При успешном входе сервер возвращает `access_token` и `refresh_token`, которые сохраняются в `localStorage`. Если вход не удался, под формой отображается сообщение об ошибке (например, "Invalid credentials"). После успешного входа пользователь автоматически перенаправляется на страницу `Dashboard`.

– `RegisterPage` — форма регистрации нового пользователя. Содержит поля для электронной почты, пароля и подтверждения пароля. На клиенте выполняется валидация: проверка формата email, минимальная длина пароля (обычно 6 символов), совпадение пароля и подтверждения. При регистрации отправляется POST-запрос на `/api/auth/register` с учётными данными.

– `DashboardPage` — личный кабинет, главная страница для авторизованного пользователя. При загрузке страницы выполняется GET-запрос к `/api/users/{userId}/resumes`, который возвращает список всех резюме текущего пользователя. Резюме отображаются в виде карточек или таблицы с основной информацией (ФИО, должность, дата создания). Для каждого резюме доступны кнопки действий: "Просмотреть", "Редактировать" (переход на `/resumes/:id/edit`), "Удалить", "Скачать PDF".

– CreateResume (используется для создания и редактирования) — форма конструктора резюме, одна из самых сложных страниц приложения. При загрузке в режиме редактирования (/resumes/:id/edit) страница получает ID резюме из параметров маршрута, отправляет GET-запрос на /api/resumes/{id} и заполняет форму существующими данными. При отправке формы все данные собираются в объект, преобразуются в JSON-строку и отправляются в поле data формы. При успешном сохранении пользователь перенаправляется на Dashboard.

2.6 Использование системы контроля версий, развертывание

Разработка велась с использованием системы контроля версий Git. Репозиторий создан на платформе GitHub под названием resume-creator-curs и сделан публичным.

Работа велась в единой ветке main. Каждый коммит сопровождался сообщением, отражающим суть изменения.

Использование Git позволило сохранить историю разработки, разделить работу по модулям через отдельные ветки и при необходимости вернуться к предыдущей версии кода.

Для развёртывания выбран подход с контейнеризацией приложения через Docker и Docker Compose, без использования облачной виртуальной машины и Terraform. Приложение состоит из двух сервисов: backend (REST API на NestJS) и frontend (SPA на Vue 3). Каждый сервис снабжен собственным Dockerfile; для локальной разработки используется docker-compose.yml со сборкой образов из исходного кода, а для продакшена — отдельный docker-compose.prod.yml, использующий заранее собранные образы с Docker Hub или GitHub Container Registry

Процесс развёртывания автоматизирован с помощью GitHub Actions (файл .github/workflows/ci.yml) и запускается при каждом пуше в ветку main:

- проверяется корректность конфигурации Docker Compose (docker compose config);

- для каждого сервиса собирается Docker-образ и публикуется в GitHub Container Registry (ghcr.io) с тегами по ветке, хешу коммита и latest;
- после успешной сборки всех образов запускается job деплоя: файл `docker-compose.prod.yaml` копируются на сервер по SCP;
- по SSH на сервере выполняется авторизация в GitHub Container Registry, скачивание (pull) новых образов и пересоздание контейнеров командой `docker compose -f docker-compose.prod.yaml up -d --force-recreate`;

Учётные данные для доступа к серверу (адрес, пользователь, приватный SSH-ключ) и токен для чтения из GitHub Container Registry хранятся в виде защищённых секретов GitHub Actions и не попадают в код репозитория.

2.7 Тестирование и верификация работоспособности

Тестирование проводилось вручную путём прямых HTTP-запросов к развёрнутому в Docker-контейнерах API с использованием Postman, а также путём анализа логов контейнеров (`docker logs`) и кода обработчиков ошибок. Проверялись следующие сценарии.

Загрузка резюме пользователя: авторизованный пользователь отправляет GET-запрос к `/api/users/1/resumes` (где 1 — его ID из JWT-токена). Сервер возвращает 200 OK и массив всех резюме этого пользователя.

Регистрация и авторизация: запрос `POST /api/auth/register` с тестовыми учётными данными (`user1@mail.ru /password1`) возвращает 200 OK

Авторизованный пользователь отправляет POST-запрос на создание резюме к `/api/resumes` с данными. Сервер возвращает 201 Created.

Клиентская часть проверялась в браузере с использованием инструментов разработчика (вкладки Network и Console): отслеживались отправляемые запросы, корректность отображения состояний загрузки и ошибок в формах.

По результатам проверки все сценарии выполняются корректно: операции CRUD над резюме и пользователями работают согласно описанной объектной модели, а ошибки валидации и авторизации возвращаются с ожидаемыми HTTP-статусами и информативными сообщениями.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано веб-приложение для создания резюме, демонстрирующее применение современных технологий веб-разработки и принципов проектирования программного обеспечения. В теоретической части работы были рассмотрены основные этапы развития интернет-технологий, изучены ключевые понятия веб-разработки, проанализированы современные подходы к созданию клиент-серверных приложений. Также был проведён анализ существующих сервисов по созданию резюме и поиску работы, что позволило определить основные требования к разрабатываемой системе и выделить её ключевые функциональные возможности.

В практической части работы последовательно решены все поставленные задачи:

- сформулированы функциональные и нефункциональные требования к веб-приложению;
- разработана объектная модель системы, включающая сущности пользователей и резюме;
- спроектирована архитектура приложения;
- реализована серверная часть приложения на базе платформы NestJS с использованием языка TypeScript;
- разработаны сервисы аутентификации пользователей, и работы с резюме;
- реализованы механизмы аутентификации и авторизации пользователей на основе JWT-токенов;
- разработана клиентская часть приложения с использованием Vue 3, TypeScript и Vue Router;
- реализован пользовательский интерфейс для просмотра, создания и редактирования резюме и его экспорта в pdf;
- выполнена контейнеризация приложения с использованием Docker и Docker Compose;

- настроены процессы непрерывной интеграции и доставки (CI/CD) на базе GitHub Actions;
- использована система контроля версий Git для фиксации этапов разработки и хранения исходного кода проекта;
- проведено тестирование функциональности приложения, подтвердившее корректность работы основных бизнес-процессов системы.

Таким образом, поставленная цель курсовой работы достигнута, а все задачи успешно выполнены. Разработанное веб-приложение обеспечивает возможность создания, просмотра и редактирования резюме.

Перспективами дальнейшего развития проекта могут стать интеграция внедрение ИИ-ассистента, размещение базы вакансий с возможностью подбора резюме под конкретные требования работодателя, интеграция с API популярных платформ по поиску работы, разделение пользователей на роли («Соискатель», «Рекрутер»).

Выполнение курсовой работы позволило закрепить теоретические знания по дисциплине «Интернет-технологии», получить практический опыт разработки современных веб-приложений и освоить инструменты, широко применяемые в профессиональной деятельности разработчиков программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) ГОСТ 19.101-2024. Единая система программной документации. Виды программ и программных документов. – Введ. 01.03.2025. – Москва : Российский институт стандартизации, 2024. – 12 с.
- 2) ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. – Взамен ГОСТ 19.101-77 ; введ. 01.01.1980. – Москва : Издательство стандартов, 1978. – 6 с.
- 3) ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем. – Введ. 01.12.2021. – Москва : Российский институт стандартизации, 2021. – 28 с.
- 4) ГОСТ 34.603-92. Информационная технология. Виды испытаний автоматизированных систем. – Введ. 01.01.1993. – Москва : Издательство стандартов, 1992. – 14 с.
- 5) ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – Введ. 01.06.2011. – Москва : Стандартинформ, 2011. – 98 с.
- 6) ГОСТ Р 56939-2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования. – Введ. 01.01.2025. – Москва : Российский институт стандартизации, 2024. – 32 с.
- 7) ГОСТ Р 50922-2006. Защита информации. Основные термины и определения. – Введ. 01.01.2007. – Москва : Стандартинформ, 2006. – 30 с.
- 8) ГОСТ 7.82-2001. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления. – Введ. 01.07.2002. – Москва : ИПК Издательство стандартов, 2002. – 12 с.
- 9) СанПиН 2.2.2/2.4.1340-03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы.

– Утверждены постановлением Главного государственного санитарного врача Российской Федерации от 30 мая 2003 г. № 118. – Москва : Минздрав России, 2003. – 42 с.

10) СанПиН 2.4.2.2821-10. Санитарно-эпидемиологические требования к условиям и организации обучения в общеобразовательных учреждениях. – Утверждены постановлением Главного государственного санитарного врача Российской Федерации от 29 декабря 2010 г. № 189. – Москва : Минздрав России, 2011. – 56 с.

11) ФАМ В. Т., ДОЛМАТОВ А. В., ЛЫУ Н. Т. СОЗДАНИЕ RESTFUL BACKEND API С ИСПОЛЬЗОВАНИЕМ NESTJS, АУТЕНТИФИКАЦИИ JWT И БАЗЫ ДАННЫХ TYPEORM ДЛЯ MSSQL. – Ассоциация выпускников и сотрудников ВВИА им. профессора НЕ Жуковского содействия сохранению исторического и научного наследия ВВИА им. профессора НЕ Жуковского КОНФЕРЕНЦИЯ: ИННОВАЦИОННЫЕ, ИНФОРМАЦИОННЫЕ И КОММУНИКАЦИОННЫЕ ТЕХНОЛОГИИ Махачкала, 01–10 октября 2023 года Организаторы: МИРЭА-Российский технологический университет.

12) Свекис Л. Л. и др. JavaScript с нуля до профи. – Питер, 2024.

13) Флэнаган, Д. JavaScript. Полное руководство : справочник по самому популярному языку программирования / Д. Флэнаган ; перевод с английского и редакция Ю. Н. Артеменко. – 7-е изд. – Санкт-Петербург : Символ-Плюс, 2021. – 1088 с. – ISBN 978-5-907203-79-2.

14) Данатарова М. К. Разработка веб-приложений: фронтенд, бэкенд и как они взаимодействуют //НАУЧНЫЙ ЭЛЕКТРОННЫЙ ЖУРНАЛ «АКАДЕМИЧЕСКАЯ ПУБЛИЦИСТИКА». – 2025. – С. 39.

15) Прокопенко, А. В. Разработка веб-приложений на Node.js / А. В. Прокопенко. – Москва : ДМК Пресс, 2022. – 412 с. – ISBN 978-5-97060-951-8.

16) Файн Я., Моисеев А. TypeScript быстро. – Питер, 2024.

17) Оберн, М. Проектирование архитектуры API / М. Оберн. – Москва : Эксмо, 2024. – 288 с. – ISBN 978-601-09-5053-5.

18) Матюшкин Б. А., Часов К. В. К вопросу создания динамических WEB-страниц //Прикладные вопросы точных наук Материалы II Международной научно-практической конференции студентов, аспирантов, преподавателей, посвященной. – 2018. – С. 223-225.

19) Дыновски, Л. Стили API. Проектирование и внедрение / Л. Дыновски, М. Дулак. – Санкт-Петербург : Питер, 2023. – 416 с. – ISBN 978-601-14-1157-8.

20) Кравченко Д. А. Микросервисная архитектура //Интерактивная наука. – 2022. – Т. 4. – №. 69. – С. 43..

21) REST API Development with Node.js / F. Doglio. – [Б. м.] : Springer Nature, 2020. – 320 с. – ISBN 978-1-4842-5964-9.

22) Позевалкин, В. В. Разработка веб-приложений на основе клиентских каркасов и библиотек : учебное пособие / В. В. Позевалкин, Н. Ф. Панова. – Москва : Ай Пи Ар Медиа, 2021. – 215 с. – ISBN 978-5-4497-0945-7.

23) Официальная документация Node.js [Электронный ресурс]. – Режим доступа: <https://nodejs.org/docs/> (дата обращения: 28.03.2026). – Загл. с экрана.

24) Официальная документация NestJS [Электронный ресурс]. – Режим доступа: <https://docs.nestjs.com/> (дата обращения: 23.06.2026). – Загл. с экрана.

25) Использование Fetch API // MDN Web Docs [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Fetch_API (дата обращения: 28.03.2026). – Загл. с экрана.

26) CORS middleware // Официальная документация Express.js [Электронный ресурс] – Режим доступа: <https://expressjs.com/ru/resources/middleware/cors.html> (дата обращения: 27.01.2026). – Загл. с экрана.

27) Building REST APIs with Express.js: практическое руководство [Электронный ресурс] – Режим доступа: <https://www.freecodecamp.org/news/build-a-rest-api-with-express-js/> (дата обращения: 28.01.2026). – Загл. с экрана.

28) Репозиторий курсовой работы «Веб-приложение для создания резюме» [Электронный ресурс] – Режим доступа:
<https://github.com/DeLineOfficial/resume-creator-curs>